Smart Pet Feeder

An IoT-Enabled Automatic Pet Feeding and Monitoring System

---

1. Problem Statement

Develop an IoT-based smart pet feeding system that automatically dispenses food according to a predefined feeding schedule. The RTC DS3231 module keeps track of the current time and triggers the feeding process at scheduled intervals. An MG996 servo motor controls the food dispensing mechanism, while the load cell and HX711 module measure the amount of food dispensed or remaining in the feeding bowl. An IR sensor can detect the presence of the pet near the feeder. The ESP32 can display feeding information on an OLED screen and optionally provide remote monitoring and control through an IoT platform. Students should design a working food container and dispensing mechanism to demonstrate automatic scheduling, portion control, sensor-based monitoring, and IoT-enabled pet care.

---

2. Background and Context

Pet owners often cannot feed their pets at fixed times, especially when they are away, at work, or busy with academics. Irregular feeding leads to overfeeding, underfeeding, or missed meals.
Most existing automatic feeders are purely timer-based: they run a motor for a fixed duration and hope the right amount falls out. The quantity then varies with food size, shape, and hopper level. This project replaces open-loop timing with sensor feedback, so the amount of food is measured rather than guessed.
The system offers several practical benefits:

- Consistent feeding: meals are dispensed on schedule without manual effort.
- Portion control: dispensing stops when the measured weight reaches the target, and leftover food in the bowl is taken into account.
- Reduced human error: missed meals and inconsistent portions caused by changes in routine are avoided.
- Remote visibility: the owner can check the next feeding time, the feeding log, and any errors from a phone or laptop, and trigger a manual feed.
- Educational value: the project integrates sensing, decision-making, actuation, mechanical design, and wireless communication into one cyber-physical system, which is the goal of the Tinkering Lab.

---

3. Challenges

Challenge	Why does it matter ?
Reliable dispensing mechanism	Food shape and size affect flow and can cause blockages. The MG996R is a positional servo, so the gate design is critical.
Accurate portion measurement	The load cell needs calibration, and vibration or falling food causes fluctuating readings. Food still in flight when the gate closes causes overshoot.
Mechanical isolation of the bowl	The load cell must carry only the bowl, with no contact between the bowl platform and the frame.
Feeding event synchronization	The same schedule slot must not trigger twice, including after a power loss or reboot.
Sensor reliability	The IR sensor can give false detections depending on ambient light and surroundings.

Power management	The servo draws current spikes that can brown out the microcontroller or disturb HX711 readings.
Fault handling	Blocked dispenser, target weight not reached, or abnormal sensor values must be detected and handled safely.

---

4. Objectives

Primary objective: design and build a working smart pet feeder that dispenses a controlled quantity of food according to a feeding schedule, with local display and remote monitoring/control.
Specific goals:

1. Interface the DS3231 RTC with the ESP32-CAM for accurate scheduling.
2. Design a food container and servo-operated dispensing mechanism.
3. Measure food dispensed and food remaining in the bowl using the load cell and HX711.
4. Implement closed-loop portion control with overshoot compensation.
5. Use the IR sensor to detect motion and trigger camera to sense pet presence and log bowl visits.
6. Display time, next feeding, target and measured quantity, and status on the OLED.
7. Implement fault handling and safety limits.
8. Provide a Wi-Fi web interface for status, feeding log, schedule view, and a "Feed Now" button.
9. Demonstrate the full system as a working physical prototype.

---

5. Portion Control and Bowl Monitoring

A simple feeder opens the servo for a fixed time (for example, "open, wait 2 s, close"). The amount of food released in that time varies. This system uses measured weight as feedback instead.

5.1 Dispensing algorithm

1. Baseline: take the average of N load-cell samples to get the starting bowl weight .
2. Leftover check: if the food already in the bowl exceeds a configurable limit, skip the feeding (or reduce the portion) and log the reason. This prevents overfeeding when the pet has not eaten.
3. Dispense: open the gate and read the filtered weight continuously.
4. Early stop: close the gate when **weight ≥ W0 + target − overshoot_margin.** The margin accounts for food still in flight and is determined empirically during calibration.
5. Settle and verify: wait for the reading to stabilize, then take the final reading **W1**. If w1 is short of the target by more than the tolerance, allow a short top-up pulse (limited number of attempts).
6. Record: **dispensed = W1 − W0,** shown on the display and logged.

5.2 Filtering

Readings are averaged over multiple samples and passed through a median filter to reject spikes from vibration and falling food. The load cell is started at startup and calibrated with a known reference mass.

5.3 Eating monitoring

The drop in bowl weight between feeding events estimates the quantity eaten. Combined with IR-detected visits, this gives a simple per-day record of feeding and eating behaviour.

5.4 Fault handling

- Maximum dispensing time: the gate closes after a fixed timeout (for example, 8 s) regardless of weight.
- Blockage detection: if the gate is open and weight does not increase for a few seconds, the system treats this as a jam, tries a short open/close "wiggle" to clear it (limited retries), and otherwise aborts.
- Error reporting: any abort is shown on the display, logged, and visible on the web interface.
- Sanity checks: implausible weight readings (for example, negative or far above the bowl capacity) cancel the feeding.

---

6. Hardware Design

The system is built around an **ESP32**, which acts as the main controller and also provides Wi-Fi support. The **DS3231 RTC** keeps accurate time for scheduled feeding, while the **MG996R servo motor** controls the gate of the food container. A **load cell with HX711** measures the food present in the bowl, and an **IR sensor** detects whether the pet is near the feeder. The **OLED** shows the current time, bowl weight, feeding status, and errors.

Part	Role
ESP32-CAM	Main controller with Wi-Fi
DS3231 RTC	Maintains feeding time
MG996R Servo	Opens/closes food gate
Load Cell + HX711	Measures bowl food weight
IR Sensor	Detects pet presence
OLED	Displays status
LiPo Supply	Stable servo power

RTC time check ▸ ESP32 checks schedule ▸ IR sensor detects pet ▸ Load cell checks bowl weight ▸ Servo opens gate ▸ Food is dispensed ▸ Target weight reached ▸ Servo closes gate ▸ Web status updated

Mechanically, the feeder uses a **gravity-fed hopper** placed above the bowl. The servo opens a small gate to release food, and the bowl is mounted on the load cell so only the bowl contents are measured. The servo is powered using a separate **LiPo supply**, while all modules share a common ground for stable operation. This design keeps the system simple, reliable, and suitable for a working lab prototype.

---

7. Software Design

Developed in the Arduino/ESP32 environment around a non-blocking state machine (timing with `millis()` instead of blocking `delay()` calls), so that Wi-Fi, display updates, and sensor reads continue during feeding.

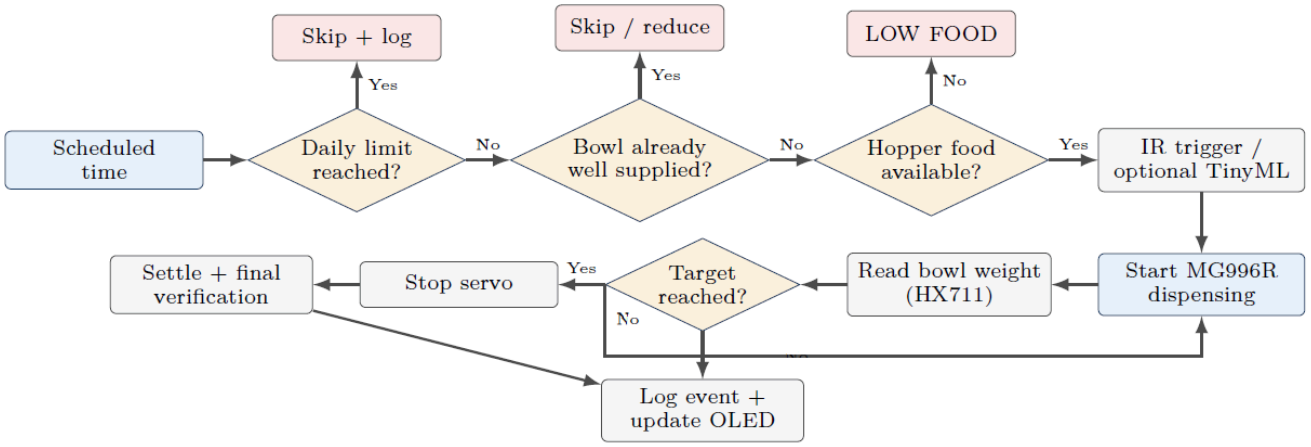
IDLE ▸ CHECK SCHEDULE ▸ PRE-FEED CHECK (baseline, leftover) ▸ DISPENSING
▼
TARGET REACHED / TIMEOUT / FAULT ◂ SETTLE & VERIFY ◂ LOG & DISPLAY
▲

Key software functions

- RTC reading and scheduling
- Servo control with gate open/close positions and jam-clearing wiggle
- HX711 reading, tare, calibration, averaging, and median filtering
- Portion-control and leftover logic
- IR sensing with debouncing
- Display driver and button-based schedule/portion setting
- Feeding log (time, target, dispensed, status, visits)
- Wi-Fi web server
- Error handling

Schedule persistence: the schedule, portion size, and the "last fed" slot and date are stored in ESP32 flash (Preferences/NVS). After a reboot or power loss, the system knows which slots have already been served and does not feed twice. The DS3231, backed by a coin cell, keeps correct time through power loss.
Setting the schedule: the schedule and portion size can be changed using the local buttons and display, or through the web interface, without reflashing.
Web interface (core feature): a lightweight page served by the ESP32 showing the current time, next feeding time, last feeding result, feeding log, current bowl weight, and error status.

8.Detailed Feeding Workflow



---

9. Scope

Minimum Viable Product (core)

- RTC-based scheduled feeding
- Servo-controlled dispensing mechanism
- Load-cell-based closed-loop portion control with overshoot compensation
- Leftover-aware feeding and dispensed/eaten logging
- IR-based pet detection and visit logging
- OLED/LCD status display .
- Fault handling (timeout, jam detection, error reporting)
- Wi-Fi web interface with status, log, and Feed Now
- Working physical container, mechanism, and bowl

Primary success criterion: the prototype reliably dispenses a specified quantity of food at a scheduled time and reports the result locally and remotely.

Possible extensions (after the MVP)

- Remote schedule editing with authentication
- Cloud dashboard and mobile notifications
- Low-food alerts
- ESP32-CAM visual monitoring and image-based pet recognition using Tiny ML(CNN)
- Multi-pet and multi-schedule support
- Stale Food Detection Based on the amount of Food present already in the container.

---

10. Timeline (6 weeks)

Weekwise Progress :-

1. Problem analysis, system design, component study, project proposal.
2. Sensor interfacing: RTC, IR sensor, load cell + HX711 (with calibration).
3. Servo control and first version of the dispensing mechanism.
4. Integration of sensors, servo, display, and schedule; portion-control algorithm.
5. Wi-Fi web interface, overshoot calibration, fault handling, debugging.
6. Final assembly, full testing, documentation, demonstration

Extensions are attempted only if the MVP, including the web interface, is complete and tested.

---

11. Workload Distribution

Contributions	Topics	Details
Harsh Thapar **Gorish Lamba** **Keerat Singh**	Embedded Software & Control	ESP32 firmware, state machine, scheduling, RTC interfacing, schedule persistence, system integration
Harsh Thapar **Gorish Lamba** **Chayan Juneja**	Sensing & Measurement	Load cell + HX711 interfacing, calibration, filtering, portion-control algorithm, leftover and eaten tracking
Keerat Singh **Chayan Juneja** **Gurnoor Singh**	Mechanical Design	Food container, servo-operated dispensing gate, bowl and load-cell mounting, jam prevention
Chayan Juneja **Keerat Singh** **Gurnoor Singh**	Electronics & Power	Circuit design and wiring, power supply and servo isolation, IR sensor integration, display and buttons interfacing
Gorish Lamba **Gurnoor Singh** **Harsh Thapar**	IoT, Testing & Documentation	Wi-Fi web interface and feeding log, fault and test cases, performance evaluation, documentation and presentation

The responsibilities are intended to divide the development work rather than isolate team members from the rest of the project. All members will participate in coding, integration, testing, debugging, documentation and final presentation. Each member will also maintain a basic understanding of the complete system.

12. Expected Outcome and Conclusion

At the end of the project, a functional Smart Pet Feeder prototype will:

- Keep accurate time with the RTC and feed automatically on schedule.
- Dispense food through a servo-operated mechanism with load-cell feedback.
- Measure food dispensed and remaining in the bowl, and avoid overfeeding.
- Detect and log pet visits with the IR sensor.
- Show feeding information on the OLED.
- Handle dispensing faults safely.
- Provide remote status, logging, and manual feeding through a Wi-Fi web interface.

The key technical feature is feedback-based portion control: the quantity of food is measured and verified, not estimated from motor time. The project demonstrates the integration of sensing, computation, control, actuation, mechanical design, and IoT communication in a real embedded system, and it leaves a clear path to further extensions such as a camera, notifications, and multi-pet support.